CSC1046: Group 8

Assignment Phase 1 - Analysis

Option B: myReversi

Group Members:

Chinedum Ogochukwu (20600)

Petra Sartori (23324986)

Robert Remizov (28889)

Lucero Dafne Sangueza Espinosa (A23100952)

Uchenna King (24522)

Matt Dela Paz (23835)

Contents

Requirements 3

Questions of Project Description 3

Initial Assumptions 3

Resolutions and Clarifications 3

Functional requirements 3

Network Programming 4

Graphic or visual elements: 4

Special algorithms: 4

Computer-player logic: 4

Scenarios 5

Primary Class List 7

CRC Cards 7

Class Diagrams 9

Use Case Descriptions 12

Use Case Diagram 16

Result of “Structured walk-through” 16

Single-player game walkthrough and identified risks with verification steps 16

Minutes of team meetings 18

Requirements

Questions of Project Description

- What constitutes a 'legal outflanking move'?
- How does the board know what the possible moves are for black/white turn?
- What is the algorithm used for each difficulty to identify how to convert the colour of the discs after a valid move?
- Should the server support multiple games at once?
- What happens if a player sends an invalid move?
- Should the server or client validate game rules?
- What happens if the player disconnects?
- How are errors displayed to the user?
- Does the server store game history?
- What technology and programming languages will be used?

Initial Assumptions

Before getting to play the game and understanding it, we had the following assumptions of the game. These assumptions represent our initial interpretation of the game.

- White moves first based on chess rules.
- There is no randomness in the game; both players are provided with information on all possible moves.
- Certain parts of the myReversi board hold higher strategic value compared to others just like in chess with the central board pieces.
- 2 players are supported in the base version of the game.
- Turn skipping occurs when no possible moves can be identified by the system.
- It is not possible for both players to have no possible move.
- Players can reconnect to the game.

Resolutions and Clarifications

After playing and understanding the game, we know it is a two-player game played on an 8x8 board. The game starts with 2 white and 2 black discs placed diagonally in the centre of the board. The game starts with the black disc moving first. A valid move outflanks opposing discs horizontally, vertically or diagonally between the newly placed disc and another disc that belongs to the current player. If a player has no legal moves, their turn is skipped. If none of the players can move, the game ends, and the player with most discs will win.

Our implementation will use a client server structure and support single player and online multiplayer modes. In single player mode, the user will play against a computer. This mode will support 3 levels of difficulty: easy, medium, and hard. Each level will determine the strategy in which the computer can select its own available legal move. In multiplayer mode, players will have usernames, create, and join lobbies. Each game lobby will start the game as soon as it has 2 players in it. The server will support multiple games at the same time, and if a player disconnects from the session, the current game ends.

The server oversees coordination and the game state, valid moves, processes disc flips, manages turns and communicates with the opponent (which will be another user or a computer). Once a user must make a move, the system will highlight any legal moves available, and players will only be able to select those allowed moves, if they choose an invalid move the game ignores it. Multiplayers can communicate using predefined messages.

Functional requirements

Our requirements are divided into the following areas:

Network Programming

- A player will be able to view available lobbies
- A player will be able to create a new public lobby
- Players will be identified by usernames
- A player will be able to choose an available public lobby to join
- Two players will be able to connect to an empty lobby to play the same game
- A lobby will only have 2 players as maximum
- The game should synchronise the board state between both players.
- A multiplayer game will begin once it has 2 players
- The server will support multiple lobbies simultaneously
- Moves made by one player should be communicated to the other player in real time.
- The server should identify whose turn it is.
- The server will validate moves and coordinate changes of the game state
- If a player disconnects during an online game, the game will end and the other player will be given the win.
- The system will allow two-way communication between players
- Players will be able to communicate using a set of predefined messages

Graphic or visual elements:

- The game will display an 8×8 Reversi game board.
- The game will display black and white discs on the board.
- The game will clearly indicate the current player's turn.
- The game will highlight or indicate possible legal moves.
- The game will visually show discs being flipped when a move is made.
- The game will display the current score for both players.
- The game will display a message when the game ends, showing the winner or if the game is a draw.
- Players will be able to start a new game or leave the current game
- The multiplayer interface will display available public lobbies and allow players to create or join them
- The single player interface will allow the player to select Easy, Medium, or Hard difficulty
- Predefined multiplayer messages will be displayed to the receiving player

Special algorithms:

- The system will determine all legal moves available to the current player.
- The system must check all eight directions from a selected position for possible captures.
- The system must identify which opponent discs should be flipped after a valid move.
- The system will update the board after a valid move.
- The system must calculate the current number of black and white discs.
- The system must determine when a player has no valid moves and must skip their turn.
- The system must determine when the game must end (no player has a legal move available).
- The system must determine the winner based on the final number of discs or determine a draw if the scores are equal

Computer-player logic:

- Single player mode will allow single human players play against a computer opponent
- The computer player will receive the legal moves available from the server.
- The computer player will automatically select and perform one of the available legal moves

- The computer player will support Easy, Medium, and Hard difficulty levels.
- Easy difficulty will select randomly from the available legal moves.
- Medium difficulty will use a simple guideline to prioritise moves such as capturing many discs or controlling strategically important positions such as corners.
- Hard difficulty will use a stronger guideline considering additional strategic factors.
- Difficulty levels will only apply to single-player games and will not affect online player-player games.
- The system should make its move automatically when it is the computer's turn.

Scenarios

1. **Charlie starts a new single-player game against a Computer**
   Current System State: Charlie has the myReversi website open.
   Informal Scenario: Charlie presses the 'New Game' button. This displays a windowed option to her with two buttons showing single player and multiplayer game modes. Charlie presses the single player game mode. Another windowed option pops up asking what difficulty rating Charlie wants to go against. Charlie chooses the medium Computer rating. The medium Computer gets readied up and the myReversi board and the 2 white discs and 2 black discs are created. A message prompts Charlie to make the first move as Black.
   Next Scenario: Charlie makes a move
2. **Charlie makes a valid move**
   Current System State: It is Charlie's turn in an ongoing myReversi game
   Informal Scenario: Charlie sees the blackened outline on possible moves they can make. Charlie presses on one of the possible valid moves. A black piece is placed, and the remaining blackened possible moves are removed from view. All affected white pieces by the new black piece are flipped to black. A message appears on screen telling Charlie that now it is the Computer's turn to play.
   Next Scenario: Computer makes a move
3. **The Computer makes a move**
   Current System State: It is the Computer's turn in an ongoing myReversi game against Charlie
   Informal Scenario: The Computer calculates what valid move to make. The Computer chooses one of the possible valid moves it can make. The move it makes depends on the difficulty of the Computer that Charlie picked. (If Charlie chose the easy difficulty, the Computer will randomly select a legal move, if Charlie chose the medium difficulty the Computer prioritises moves that capture more discs and strategically valuable positions such as corners. If Charlie chooses hard difficulty the computer uses additional strategic factors when selecting a legal move). A white piece is placed in the chosen Computer's place. All affected black pieces by the new white piece are flipped to white. A message appears on screen telling Charlie that it is his turn to play.
   Next Scenario: Charlie makes a move
4. **Charlie makes an invalid move**
   Current System State: It is Charlie's turn in an ongoing myReversi game.

Informal Scenario: Charlie is shown in a blackened outline the possible moves that black can make against black. Charlie presses one of the non-blackened outlines. The game ignores Charlie's invalid move and waits for Charlie to press a possible move.

Next Scenario: Charlie makes a valid move

5. **Charlie has no possible moves**

   Current System State: It is Charlie's turn in an ongoing myReversi game.

   Informal Scenario: The game calculates for possible moves Charlie can make. It finds no possible moves that Charlie can make. The game prompts Charlie to tell them that 'Turn has been skipped.' The game continues skipping Charlie's turn and waits for the Computer to make a move.

   Next Scenario: **Computer** makes a move

6. **Both players have no possible moves**

   Current System State: There is an ongoing myReversi game.

   Informal Scenario: The game has evaluated that both Charlie and the Computer have no possible moves. The game concludes. The game calculates the number of black pieces and the number of white pieces. If black has more pieces, the game displays the message that Black has won this game of myReversi.

7. **Charlie starts a new multiplayer game against an online player**

   Current System State: Charlie has the myReversi website open

   Informal Scenario: Charlie presses the 'New Game' button. This displays a windowed option to her with two buttons showing single player and multiplayer game modes. Charlie presses the multiplayer game mode. This directs Charlie to the login page.

   Next Scenario: Charlie makes an account

8. **Charlie enters username**

   Current System State: Charlie is on the myReversi website and introduces his name

   Informal Scenario: Charlie has selected multiplayer mode and is asked to input his name. Charlie enters the username and the game accepts it. The game then displays the available multiplayer lobbies and gives Charlie the option to join an existing lobby or create a new lobby

   Next Scenario: Charlie creates an online multiplayer lobby.

9. **Charlie creates an online multiplayer lobby**

   Current System State: Charlie is on the multiplayer lobby screen

   Informal Scenario: Charlie is prompted with join lobby and create lobby. Charlie presses create lobby. Charlie is brought to the waiting room. Charlie tells Bill to select the public lobby with Charlie's name. Bill joins Charlie's lobby and the game automatically starts.

   Next Scenario: Charlie plays a multiplayer game with Bill

10. **Bill joins Charlie's multiplayer lobby**

    Current System State: Bill loaded myReversi website and is waiting to join the lobby

    Informal Scenario: Bill sees a public lobby with Charlie's name. Bill chooses to join Charlie's lobby and successfully enters Charlie's lobby. Game automatically starts.

    Next Scenario: Charlie plays a multiplayer game with Bill.

11. **Bill sends Charlie a predefined message**

    Current System State: Bill and Charlie are in an ongoing myReversi multiplayer game

    Informal Scenario: Bill presses the message chat button on the screen. The screen displays multiple preset messages ranging from 'Good luck, have fun' to 'This game is over!'. Bill presses one of the messages. Charlie sees a message icon appear with Bill's message.

    Next Scenario: Bill and Charlie play their multiplayer game.

12. Charlie disconnects from the multiplayer game

Current System State: Bill and Charlie are in an ongoing myReversi multiplayer game

Informal Scenario: Charlie suddenly disconnects from the server and loses connection. The game prompts Bill notifying him Charlie has disconnected and that he has won the game.

Primary Class List

- **Network:** ReversiServer, LobbyManager, GameManager, Lobby
- **Game:** Game, Player, ComputerPlayer, Board, Move, MoveValidator, TurnManager

CRC Cards

Class	ReversiServer
Responsibilities	- Managing multiple games simultaneously - Start and end games - Handle player disconnections - Calculate the game score - Notify players when a game has ended - Handle player disconnections
Collaborators	LobbyManager, GameManager, Lobby, Player, MessageHandler

Class	LobbyManager
Responsibilities	- Create multiplayer lobbies - Manage available/open lobbies - Allow players to create a lobby - Allow players to join a lobby - Connect players to the appropriate lobby
Collaborators	Lobby, Player, GameManager, ReversiServer

Class	GameManager
Responsibilities	- Coordinating the game - Manage the state of an individual game - Maintaining authoritative game state - Managing turns - Ending games - Synchronising players
Collaborators	ReversiServer, Player, Board, MoveValidator, TurnManager, Lobby

Class	Lobby
Responsibilities	- Stores the player count - Stores the active players - Allow players to be kicked from the lobby - Store the player with the top score - Outputs the player with the top Score - Store the lobby ID - Output the player count - Output the lobby ID

	- Allow players to join and leave the lobby
Collaborators	Player, LobbyManager, GameManager

Class	Game
Responsibilities	- Represent an individual match - Tracks current game state (game is active or game is finished) - Maintain only 2 players - Determine whether the winner or draw when the game ends
Collaborators	Player, GameManager, Board, TurnManager, MoveValidator

Class	ComputerPlayer
Responsibilities	- Represent the computer player - Receive the available legal moves from the server - Select a legal move based on difficulty - Chooses a valid move when it is the computer player's turn
Collaborators	MoveValidator, TurnManager, Move, Game, Board,

Class	Player
Responsibilities	- Represent a player in the game - Join a lobby - Leave a lobby - Make moves - Maintain the player’s username - Maintain the player’s score - Maintain the player’s ID - Take part in a game
Collaborators	Lobby, Board, MoveValidator, TurnManager, GameManager, MessageHandler

Class	Board
Responsibilities	- Representing the 8x8 board - Storing the disc positions - Updating board state - Placing discs - Flipping discs
Collaborators	MoveValidator, Player, TurnManager, GameManager

Class	Move
Responsibilities	- Represent a move made during a game - Store the position where a disc is placed - Store the player making a move - Store the discs affected by the move
Collaborators	Board, MoveValidator, Player

Class	MoveValidator

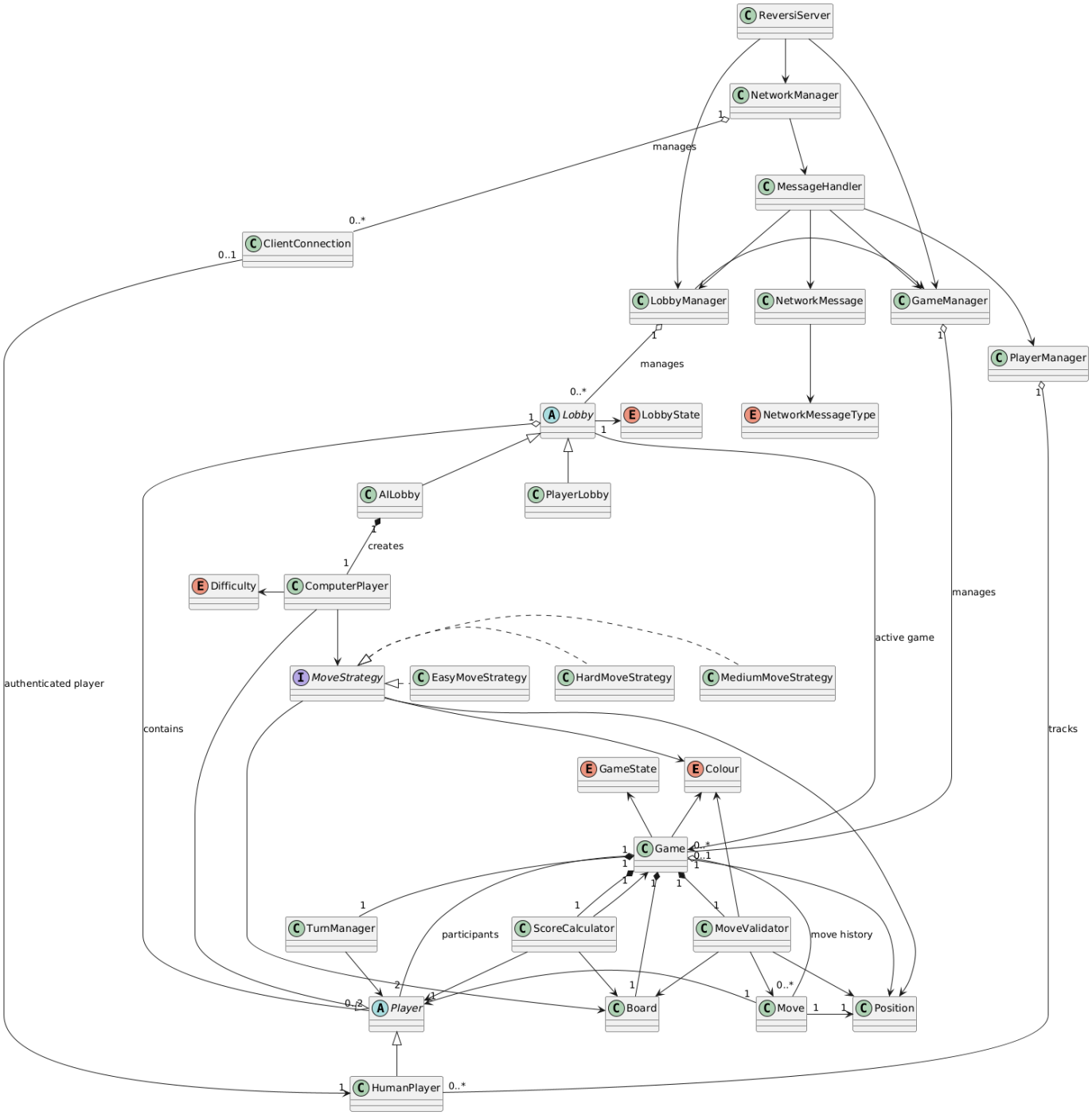
Responsibilities	- Determine all legal moves* - Check horizontal directions - Check vertical directions - Check diagonal directions - Identify opponent discs that can be captured - Reject moves that do not outflank an opponent disc
Collaborators	Board, Colour, Player, TurnManager

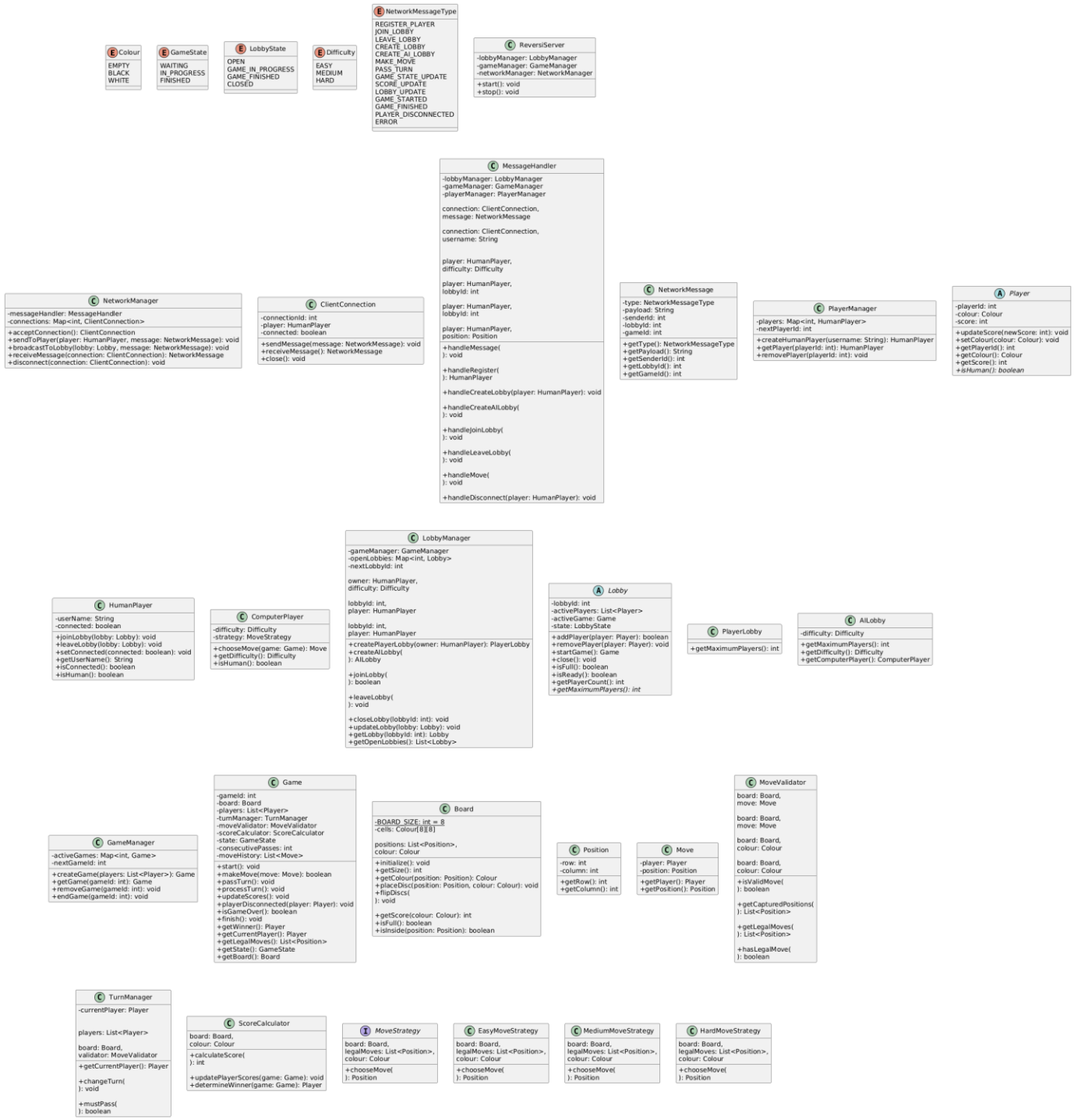
*= A legal move stipulates that when a newly placed disc creates a horizontal, vertical or diagonal line containing one or more opponent discs followed by another disc belonging to the current player

Class	TurnManager
Responsibilities	- Track which player is currently due to move - Change the current player's turn - Manage the order of turns - Skip a player's turn when they have no valid moves - Start the game with the black player taking the first move - Update the player's score
Collaborators	Player, GameManager, MoveValidator, Board

Class Diagrams

- For the diagrams to be viewable in document, class functions and attributes had to be separated from relations diagram
- The combined diagram is viewable at this link: https://drive.google.com/file/d/1w6lYpq1Gf5maJA0bHwRtQGxu0Ke0TZU-/view?usp=drive_link


ReversiServer
NetworkManager
manages
MessageHandler
ClientConnection
LobbyManager
NetworkMessage
GameManager
PlayerManager
manages
Lobby
LobbyState
NetworkMessageType
AILobby
PlayerLobby
creates
Difficulty
ComputerPlayer
active game
manages
MoveStrategy
EasyMoveStrategy
HardMoveStrategy
MediumMoveStrategy
authenticated player
contains
tracks
GameState
Colour
Game
TurnManager
participants
ScoreCalculator
MoveValidator
move history
Player
Board
Move
Position
HumanPlayer


Colour
EMPTY
BLACK
WHITE
GameState
WAITING
IN_PROGRESS
FINISHED
LobbyState
OPEN
GAME_IN_PROGRESS
GAME_FINISHED
CLOSED
Difficulty
EASY
MEDIUM
HARD
NetworkMessageType
REGISTER_PLAYER
JOIN_LOBBY
LEAVE_LOBBY
CREATE_LOBBY
CREATE_AI_LOBBY
MAKE_MOVE
PASS_TURN
GAME_STATE_UPDATE
SCORE_UPDATE
LOBBY_UPDATE
GAME_STARTED
GAME_FINISHED
PLAYER_DISCONNECTED
ERROR
ReversiServer
-lobbyManager: LobbyManager
-gameManager: GameManager
-networkManager: NetworkManager
+start(): void
+stop(): void
NetworkManager
-messageHandler: MessageHandler
-connections: Map<int, ClientConnection>
+acceptConnection(): ClientConnection
+sendToPlayer(player: HumanPlayer, message: NetworkMessage): void
+broadcastToLobby(lobby: Lobby, message: NetworkMessage): void
+receiveMessage(connection: ClientConnection): NetworkMessage
+disconnect(connection: ClientConnection): void
ClientConnection
-connectionId: int
-player: HumanPlayer
-connected: boolean
+sendMessage(message: NetworkMessage): void
+receiveMessage(): NetworkMessage
+close(): void
MessageHandler
-lobbyManager: LobbyManager
-gameManager: GameManager
-playerManager: PlayerManager
connection: ClientConnection,
message: NetworkMessage
connection: ClientConnection,
username: String
player: HumanPlayer,
difficulty: Difficulty
player: HumanPlayer,
lobbyId: int
player: HumanPlayer,
lobbyId: int
player: HumanPlayer,
position: Position
+handleMessage(
): void
+handleRegister(
): HumanPlayer
+handleCreateLobby(player: HumanPlayer): void
+handleCreateAILobby(
): void
+handleJoinLobby(
): void
+handleLeaveLobby(
): void
+handleMove(
): void
+handleDisconnect(player: HumanPlayer): void
NetworkMessage
-type: NetworkMessageType
-payload: String
-senderId: int
-lobbyId: int
-gameId: int
+getType(): NetworkMessageType
+getPayload(): String
+getSenderId(): int
+getLobbyId(): int
+getGameId(): int
PlayerManager
-players: Map<int, HumanPlayer>
-nextPlayerId: int
+createHumanPlayer(username: String): HumanPlayer
+getPlayer(playerId: int): HumanPlayer
+removePlayer(playerId: int): void
Player
-playerId: int
-colour: Colour
-score: int
+updateScore(newScore: int): void
+setColour(colour: Colour): void
+getPlayerId(): int
+getColour(): Colour
+getScore(): int
+isHuman(): boolean
HumanPlayer
-userName: String
-connected: boolean
+joinLobby(lobby: Lobby): void
+leaveLobby(lobby: Lobby): void
+setConnected(connected: boolean): void
+getUserName(): String
+isConnected(): boolean
+isHuman(): boolean
ComputerPlayer
-difficulty: Difficulty
-strategy: MoveStrategy
+chooseMove(game: Game): Move
+getDifficulty(): Difficulty
+isHuman(): boolean
LobbyManager
-gameManager: GameManager
-openLobbies: Map<int, Lobby>
-nextLobbyId: int
owner: HumanPlayer,
difficulty: Difficulty
lobbyId: int,
player: HumanPlayer
lobbyId: int,
player: HumanPlayer
+createPlayerLobby(owner: HumanPlayer): PlayerLobby
+createAILobby(
): AILobby
+joinLobby(
): boolean
+leaveLobby(
): void
+closeLobby(lobbyId: int): void
+updateLobby(lobby: Lobby): void
+getLobby(lobbyId: int): Lobby
+getOpenLobbies(): List<Lobby>
Lobby
-lobbyId: int
-activePlayers: List<Player>
-activeGame: Game
-state: LobbyState
+addPlayer(player: Player): boolean
+removePlayer(player: Player): void
+startGame(): Game
+close(): void
+isFull(): boolean
+isReady(): boolean
+getPlayerCount(): int
+getMaximumPlayers(): int
PlayerLobby
+getMaximumPlayers(): int
AILobby
-difficulty: Difficulty
+getMaximumPlayers(): int
+getDifficulty(): Difficulty
+getComputerPlayer(): ComputerPlayer
GameManager
-activeGames: Map<int, Game>
-nextGameId: int
+createGame(players: List<Player>): Game
+getGame(gameId: int): Game
+removeGame(gameId: int): void
+endGame(gameId: int): void
Game
-gameId: int
-board: Board
-players: List<Player>
-turnManager: TurnManager
-moveValidator: MoveValidator
-scoreCalculator: ScoreCalculator
-state: GameState
-consecutivePasses: int
-moveHistory: List<Move>
+start(): void
+makeMove(move: Move): boolean
+passTurn(): void
+processTurn(): void
+updateScores(): void
+playerDisconnected(player: Player): void
+isGameOver(): boolean
+finish(): void
+getWinner(): Player
+getCurrentPlayer(): Player
+getLegalMoves(): List<Position>
+getState(): GameState
+getBoard(): Board
Board
-BOARD_SIZE: int = 8
-cells: Colour[8][8]
positions: List<Position>,
colour: Colour
+initialize(): void
+getSize(): int
+getColour(position: Position): Colour
+placeDisc(position: Position, colour: Colour): void
+flipDiscs(
): void
+getScore(colour: Colour): int
+isFull(): boolean
+isInside(position: Position): boolean
Position
-row: int
-column: int
+getRow(): int
+getColumn(): int
Move
-player: Player
-position: Position
+getPlayer(): Player
+getPosition(): Position
MoveValidator
board: Board,
move: Move
board: Board,
move: Move
board: Board,
colour: Colour
board: Board,
colour: Colour
+isValidMove(
): boolean
+getCapturedPositions(
): List<Position>
+getLegalMoves(
): List<Position>
+hasLegalMove(
): boolean
TurnManager
-currentPlayer: Player
players: List<Player>
board: Board,
validator: MoveValidator
+getCurrentPlayer(): Player
+changeTurn(
): void
+mustPass(
): boolean
ScoreCalculator
board: Board,
colour: Colour
+calculateScore(
): int
+updatePlayerScores(game: Game): void
+determineWinner(game: Game): Player
MoveStrategy
board: Board,
legalMoves: List<Position>,
colour: Colour
+chooseMove(
): Position
EasyMoveStrategy
board: Board,
legalMoves: List<Position>,
colour: Colour
+chooseMove(
): Position
MediumMoveStrategy
board: Board,
legalMoves: List<Position>,
colour: Colour
+chooseMove(
): Position
HardMoveStrategy
board: Board,
legalMoves: List<Position>,
colour: Colour
+chooseMove(
): Position

Use Case Descriptions

1. Use Case: Start Single Player Game

USE CASE 1	Start Single Player Game	
Goal in Context	User starts game with computer opponent	
Scope & Level	MyReversi System, User goal	
Preconditions	User loaded the game, the user wants to play on their own	
Success End Condition	Single player game starts	
Failed End Condition	Single player game does not start	
Primary, **Secondary Actors**	Player None	
Trigger	User wants to play on their own	
DESCRIPTION	**Step**	**Action**
	1	Player chooses to start single player mode
	2	System asks the player to choose a level (easy, medium and hard)
	3	Player selects difficulty level
	4	System starts a game between player and computer opponent using the chosen difficulty level
	5	System assigns the black and white discs to the player and the computer opponent
	6	Game starts with white and black discs placed diagonally at the centre of the board
	7	Game begins with black discs participant taking the first turn
EXTENSIONS	**Step**	**Branching Action**
	4a	The game fails to start

	4a1	The system informs the player that the game could not start and offers to start another game
	4a2	Resume at step 2
	4b	The computer opponent is not responsive
	4b1	The system informs the user that the game could not start and offers to start another game
	4b2	Resume at step 2

2. Use Case: Start multiplayer game

USE CASE 2	Start multiplayer game	
Goal in Context	User starts multiplayer game to play against a human opponent	
Scope & Level	MyReversi System, User goal	
Preconditions	User loaded the game, the user wants to play with a human opponent	
Success End Condition	Multiplayer game starts	
Failed End Condition	Multiplayer game does not start	
Primary, **Secondary Actors**	Local player Remote player	
Trigger	User wants to play with a human player	
DESCRIPTION	**Step**	**Action**
	1	Local player chooses to start multiplayer mode
	2	The system shows the available lobbies with an option to create a new lobby
	3	Local player chooses to create a new public lobby
	4	System creates public lobby and allows other players to join it
	5	Remote player joins the lobby of the local player

	6	The system assigns black and white discs to the participants
	7	Game begins with discs with the black disc participant making a move first
EXTENSIONS	**Step**	**Branching Action**
	4a	The lobby could not be created
	4a1	The system informs the local player
	4a2	Return at step 2
	5a	The local player leaves the lobby before another remote player joins
	5a1	The system closes the lobby
	5b	The remote player disconnects before the game starts
	5b1	The system goes back to waiting for another remote player to join

3. Use Case: Make a move

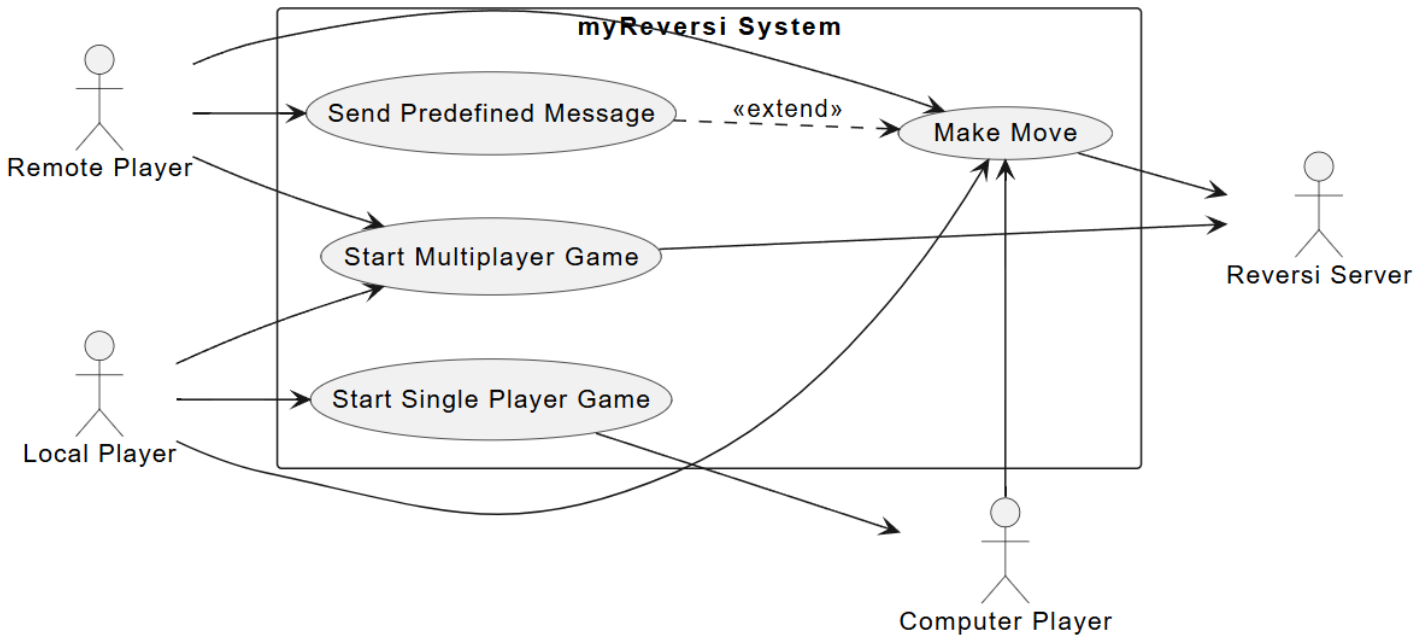
USE CASE 3	Make a move	
Goal in Context	A user makes a move during a game	
Scope & Level	MyReversi, User goal	
Preconditions	A game has been started	
Success End Condition	A user makes a legal move	
Failed End Condition	The player is interrupted before the player can make a legal move	
Primary, **Secondary Actors**	Local player, remote player None	
Trigger	User wants to make a move on their turn	
DESCRIPTION	**Step**	**Action**
	1	player's turn starts
	2	The system determines the legal moves available to the player

	3	The system shows use their possible valid moves
	4	Player selects a valid move
	5	System plays the valid move
	6	Turn is passed to the other player
EXTENSIONS	**Step**	**Branching Action**
	4a	Player attempts to select an invalid move
	4a1	System displays message "Not a valid move"
	4a2	The system leaves the board unchanged
	4a3	Player remains on their turn
	4a4	Returns at step 4

4. Use Case: Computer player turn

USE CASE 4	Computer player turn	
Goal in Context	The computer player takes its turn	
Scope & Level	MyReversi	
Preconditions	A game has been started	
Success End Condition	Computer player makes a valid move	
Failed End Condition	Game terminated, computer player algorithm fails	
Primary, **Secondary Actors**	Local player, Computer player None	
Trigger	It is the computer player's turn	
DESCRIPTION	**Step**	**Action**
	1	Computer player's turn starts
	2	Move validator checks for valid moves
	3	Computer calculates appropriate move based on difficulty
	4	System makes the move

Use Case Diagram



Result of "Structured walk-through"

Single-player game walkthrough and identified risks with verification steps

Step	Action	Classes	Use Case ID	Vulnerability/Verification
0	Human player selects single-player mode	Player, GameManager	1	**Risk:** Wrong mode loaded. **Verify:** Start the game multiple times choosing different modes.
1	Human player selects Easy, Medium or Hard difficulty.	ComputerPlayer, MoveStrategy	1	**Risk:** Wrong difficulty applied. **Verify:** Confirm selected strategy is used by the computer algorithm.
2	System generates the game and initial 8 x 8 board.	GameManager, Board	1	**Risk:** Incorrect setup. **Verify:** Check colour and placement of the four centre discs and dimensions of the board.
3	Black is assigned the first turn.	TurnManager, Player	1	**Risk:** White player starts first. **Verify:** Confirm Black starts first.
4	System determines all legal moves for the human player.	MoveValidator, Board	3	**Risk:** Legal moves not identified correctly or missed. **Verify:** Test all 8 directions while evaluating legal moves.
5	Legal positions displayed for the Human player.	MoveValidator, Board	3	**Risk:** Legal moves not identified correctly or missed.

				Verify: Test all 8 directions while evaluating legal moves.
6	Human player selects a move.	Player, Board	3	**Risk:** Human selects illegal position. **Verify:** Illegal move doesn't change the board.
7	System validates selected move.	MoveValidator, GameManager	3	**Risk:** Illegal move accepted. **Verify:** Rejection of illegal moves.
8	Disc is placed and opponent's affected discs are identified.	Board, MoveValidator, Colour	3	**Risk:** Wrong discs are captured. **Verify:** Test captures for all 8 directions.
9	Captured discs are flipped and the score is updated.	Board, Colour, ScoreCalculator	3	**Risk**: Incorrect score incrementation. **Verify**: Recount board after the move was executed.
10	Turn is passed to Computer player.	TurnManager, GameManager	3 and 4	**Risk**: Turn is not passed/Incorrect turn state. **Verify**: Confirm Computer is now an active player.
11	Computer receives a set of its legal moves.	MoveValidator, ComputerPlayer	4	**Risk**: Computer receives a set of wrong legal moves. **Verify**: Compare the received set with validator output.
12	Computer selects a move based on difficulty selected.	ComputerPlayer, MoveStrategy	4	**Risk**: Computer selects illegal/ inappropriate move for the selected difficulty. **Verify**: Validate chosen move before it's executed on the board.
13	Computer move is applied and discs are flipped.	Board, MoveValidator, Colour	4	Risk: Incorrect board update. **Verify:** Compare expected and actual board state.
14	Turn is passed back to the Human player.	TurnManager, GameManager	3 and 4	**Risk**: Turn is not passed/Incorrect turn state. **Verify**: Confirm Human is now an active player.
15	If a player cannot make any legal moves, their turn is skipped.	MoveValidator, TurnManager	3	**Risk:** Game ends too early. **Verify:** Only skip the player if the opponent can still make legal moves.
16	If both players are out of legal moves, the game ends.	MoveValidator, GameManager	3	**Risk:** Game ends too early or doesn't end at all. **Verify**: Confirm both move sets are empty.
17	Final score is calculated and winner/draw displayed.	Board, ScoreCalculator, GameManager	3	**Risk:** Wrong result calculated/displayed. **Verify:** Compare result with final disc count.

Multiplayer note: Multiplayer mode has the same core game flow, but starts with Use Case 2: Start Multiplayer Game instead of Use Case 1. This replaces the computer with a remote player and introduces classes such as

LobbyManager, Lobby, NetworkMessage, and MessageHandler. Risks associated with these classes and use cases are lobby capacity, synchronisation issues, server-side move validation and player disconnection.

Minutes of team meetings

16/09/26 | 12:40 - 2pm

Location: DCU St. Patrick's Library

Objective for today: Distribute roles for the team and assign tasks for members to get started on.

Attendance: Matt, Chinedum, Petra, Lucero, Robert and Uchenna

- Established a group meeting place in DCU Library.
- Following an agile team. Group member roles were delegated appropriately for each member.
- Colored server made by Chinedum to handle announcements, links etc.
- Established we would use PlantUMLllama3.2:3b for our use case diagrams

Lucero - Scrum Master (facilitates the group and keeps the process on track)

Matt - Scribe (useful for recording decisions, but not an official Scrum accountability) + Timekeeper (useful for the activity)

Petra - Product Owner (decide priorities/ what needs to be achieved)

Robert - Sceptic (challenge ideas) + Artist (UML case diagrams)

Uchenna - Developer

Chinedum – Developer

Robert and Uchenna delegated tasks to come up with requirements

Petra assigning tasks and their relating priority on the Jira Board

Chinedum working on the Colourord Organisation and Management of our project

Matt and Lucero working on the primary class list

23/09/26 | 12:00 - 12:50

Location: DCU St. Patrick's Library

Objective for today: Updates on everyone's progress, detailing to group the same design prospect we want to achieve. Ensuring everyone is on the same page with what we are making.

Attendance: Matt, Chinedum, Petra, Lucero, Robert and Uchenna

- Scrum master delegated task of filling in the word document with the questions, assumptions and resolutions for the requirements.
- Uchenna, Chinedum, Petra, Matt and Robert filled in the document.
- Chinedum created the word document and make it accessible to all members.
- Brainstormed ideas to further enhance the game such as communication system, speed game mode with time as a constraint to incentivise fast paced gameplay.
- Editing and removing unnecessary points.
- Further development of non-concise points.
- Lucero comparing the points in the document versus the description of the project.

30/09/26 | 12:00 - 12:50

Location: DCU L129

Objective for today: Make the requirements, scenarios, use cases, CRCs and class diagram consistent with one another

Attendance: Matt, Chinedum, Lucero, Robert and Uchenna

- Chinedum and Uchenna working on the Class Diagrams
- Robert and Lucero working on the Use Case Diagrams
- Matt and Petra working on the CRC Cards
- Matt writing up the minutes.